# SOMNGUARD

## Propuesta Técnica del Software

**Estado:** En progreso **Fecha:** 2026-08-19

## Contenido


- 1. Introducción
- 2. Descripción general de la propuesta
- 3. Arquitectura de la solución propuesta
- 4. Stack tecnológico propuesto
- 5. Diseño de la solución
- 6. Metodología de desarrollo
- 7. Plan de trabajo y cronograma
- 8. Recursos requeridos
- 9. Estimación de costos
- 10. Riesgos y plan de mitigación
- 11. Entregables
- 12. Capacitación, soporte y mantenimiento
- 13. Condiciones y supuestos


# 1. Introducción

### 1.1 Propósito del documento

El propósito de este documento es presentar la propuesta técnica para el desarrollo del software **SomnGuard**: la solución propuesta, la arquitectura, el stack tecnológico, el diseño de la solución, la metodología de trabajo, los recursos y costos estimados, los riesgos y los entregables.

Esta propuesta constituye la base técnica sobre la cual se desarrollará el proyecto y se alinea con la propuesta inicial, el documento de arquitectura, el informe de diseño de software y el SRS.

### 1.2 Alcance

La propuesta cubre:

- El software de la plataforma SomnGuard (backend, portal web y aplicación móvil).
- El software embebido en el dispositivo de monitoreo (edge) y su integración con la plataforma.
- La infraestructura de despliegue, el proceso de CI/CD y la estrategia de pruebas.

Quedan fuera del alcance de esta propuesta: el diseño de hardware del dispositivo, la fabricación, las comunicaciones con terceros (aseguradoras, autoridades) y los servicios legales asociados a la operación del sistema.

### 1.3 Documentos de referencia

| Documento | Ubicación |
|---|---|
| Propuesta inicial | 01-project-proposal/ |
| Documento de arquitectura | ../05-architecture/architecture-document.md |
| Informe de diseño de software | ../05-architecture/software-design-report.md |
| ADR-001 (stack del backend) | ../05-architecture/decisions/records/ADR-001-backend-java-spring-boot.md |
| Metodología adoptada | ../00-documentation-governance/adopted-methodology.md |
| Estrategia de CI/CD | ../10-devops/ci-cd-strategy.md |
| Estrategia de pruebas | ../11-quality-assurance/test-strategy.md |
| Modelo de datos vigente | ../06-data-architecture/02-modules-entities.md |

---

## 2. Descripción general de la propuesta

SomnGuard es un sistema de detección de fatiga y somnolencia al volante que combina **procesamiento en el borde (edge)** y una **plataforma en la nube**:

- **Dispositivo SomnGuard (edge)**: cámara y procesador embebido (Raspberry Pi) instalados en el vehículo. Analiza en tiempo real el comportamiento del conductor (parpadeo, cierre prolongado de ojos, movimientos de cabeza) mediante algoritmos de visión en Python. Ante un estado de riesgo, reproduce una alerta sonora inmediata y registra el evento con evidencia multimedia.
- **Plataforma SomnGuard (cloud)**: backend monolito modular en Java/Spring Boot con PostgreSQL, portal web en React JS y aplicación móvil en React Native. Almacena los eventos sincronizados, permite su consulta y análisis (línea de tiempo, métricas, reportes con resumen IA) y gestiona cuentas, dispositivos y catálogos.

La propuesta contempla el desarrollo completo de ambos entornos, la integración entre ellos (sincronización asíncrona con soporte offline), la infraestructura de despliegue y el proceso de calidad.

### 2.1 Objetivos de la propuesta

1. Entregar un sistema funcional de detección de somnolencia con alertas inmediatas en el vehículo.
2. Proveer una plataforma de consulta y análisis de eventos con evidencia verificable.
3. Mantener un código mantenible, probado y desplegable mediante CI/CD.
4. Entregar la documentación técnica completa alineada a la gobernanza del proyecto.

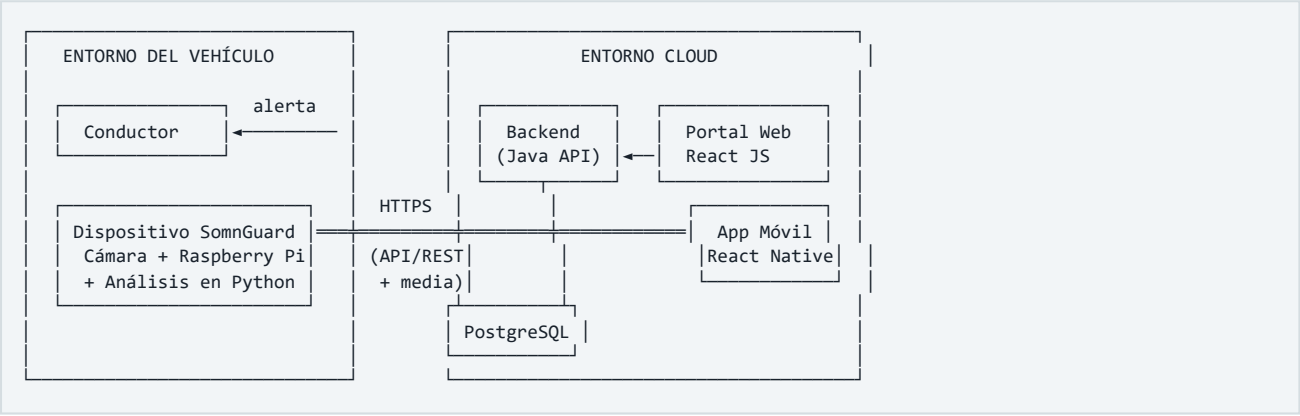
---

# 3. Arquitectura de la solución propuesta

## 3.1 Estilo arquitectónico

- **Híbrida edge-cloud**: el dispositivo opera de forma autónoma (detección y alertas locales) y sincroniza de forma asíncrona con la plataforma. No depende de conectividad para su función crítica.
- **Monolito modular** en el backend (Java/Spring Boot): una sola aplicación desplegable, organizada internamente en módulos independientes con arquitectura hexagonal (puertos y adaptadores): `application/port/{in,out}`, `application/usecase`, `domain/{model,service}`, `adapter/in/{web,amqp}`, `adapter/out/{persistence,storage}`; `platform` (errores, logging, observabilidad) va **fuera de los módulos**, al nivel de `com.somnguard` (ver ADR-002).

## 3.2 Vista general




## 3.3 Justificación técnica

| Criterio | Decisión | Justificación |
|---|---|---|
| Procesamiento en el edge | Raspberry Pi + Python | Detección en tiempo real sin depender de conectividad; latencia mínima para la alerta al conductor. |
| Backend | Java 21 + Spring Boot 4.1.1 + Maven | Madurez, ecosistema, rendimiento y productividad (ver ADR-001). |
| Monolito modular | Un artefacto, 6 módulos | Menor complejidad operativa que microservicios para un sistema de tamaño medio (ver ADR-001). |
| Base de datos | PostgreSQL + Liquibase | Motor relacional robusto y migraciones versionadas de esquema. |
| Portal web | React JS | SPA responsive de fácil mantenimiento y amplio ecosistema. |
| Aplicación móvil | React Native | Desarrollo multiplataforma (iOS/Android) con una sola base de código. |
| CI/CD | GitHub Actions | Automatización del flujo por entornos (develop → qa → main). |

# 4. Stack tecnológico propuesto

| Componente | Tecnología | Versión | Propósito |
|---|---|---|---|
| Backend | Java (LTS) | 21 | Lenguaje del backend |
| Backend | Spring Boot | 4.1.1 | Framework de la API |

| Componente | Tecnología | Versión | Propósito |
|---|---|---|---|
| Backend | Maven | 3.9+ | Gestión de dependencias y build |
| Backend | Spring Security + JWT | 7.x (emparejado con Spring Boot 4.1.1) | Autenticación y autorización |
| Base de datos | Liquibase | 4.x | Migraciones versionadas de esquema (repo `somnguard-db`) |
| Base de datos | PostgreSQL | 16+ | Almacenamiento relacional |
| Portal web | React JS | 18+ | Aplicación web (SPA) |
| App móvil | React Native | 0.7x | Aplicación móvil multiplataforma (versión a fijar en la iteración técnica) |
| Edge | Python | 3.11+ | Análisis de visión y agente de sincronización |
| Edge | OpenCV / bibliotecas de visión | — | Procesamiento de imágenes (a validar en iteración de edge) |
| Edge | Raspberry Pi OS | — | Sistema operativo del dispositivo |
| Multimedia | Almacenamiento de objetos (S3-compatible) | — | Evidencia de eventos (imágenes/video) |
| CI/CD | GitHub Actions | — | Pipelines por entorno |
| Documentación API | OpenAPI 3 / Swagger | — | Contrato y documentación de la API |
| Pruebas | JUnit 5 + Mockito; Jest; pytest | — | Pruebas unitarias y de integración |

> Las versiones específicas se fijan en la iteración de habilitación técnica; las listadas son las vigentes al momento de esta propuesta.

---

# 5. Diseño de la solución

## 5.1 Módulos del backend

| Módulo | Responsabilidad | Entidades |
|---|---|---|
| Security | Cuentas, roles, permisos, autenticación y auditoría | user, role, module, feature, role_feature, user_role, password_reset_request, audit_login |
| Parameterization | Catálogos configurables | event_category, severity, media_type, sound_pattern, event_type |
| Device Management | Ciclo de vida del dispositivo y configuración | device, device_assignment, device_config |
| Telemetry Service | Eventos, evidencia y alertas | event, evidence, alert_log |
| Monitoring | Notificaciones de eventos críticos | notification |
| Analytics | Línea de tiempo, métricas, resumen IA y reportes | vistas/reportes derivados (sin entidades transaccionales; ver ADR-003) |

### 5.2 Modelo de datos

Modelo relacional de **20 entidades** en PostgreSQL, gestionado con Liquibase. Fuentes:

- Modelo entidad-relación
- Módulos y entidades
- Diccionario de datos

### 5.3 API

API REST con diseño versionado (`/api/v1/...`) documentada en OpenAPI. Ver diseño de API.

### 5.4 Integración dispositivo ↔ plataforma

- Sincronización asíncrona de eventos por lotes con idempotencia (ID único por evento).
- Autenticación del dispositivo mediante API key almacenada como hash.
- Descarga de configuración básica del dispositivo (patrón de sonido, sensibilidad).
- Transmisión de evidencia multimedia por partes, con reintentos y cola local ante fallas.

### 5.5 Interfaces

- **Portal web**: dashboard, dispositivos, línea de tiempo de eventos, detalle con evidencia, métricas, reportes, notificaciones y administración.
- **App móvil**: dispositivos, eventos, notificaciones y configuración.
- **Dispositivo**: interfaz acústica (patrones de sonido configurables).

Detalle en informe de diseño sección 7.

---

## 6. Metodología de desarrollo

- **Metodología ágil (SCRUM adaptado)**, según adopted-methodology.md: historias de usuario, iteraciones (sprints) y reuniones de planificación/revisión documentadas en actas.
- **Flujo de ramas**: `hu-<repo>-###-dev → develop`, `hu-<repo>-###-qa → qa`, `hu-<repo>-###-main → main`; propagación de cambios entre entornos por copia/cherry-pick, sin merges directos entre `develop`/`qa`/`main`.
- **CI/CD**: pipeline de integración continua y despliegue por entorno (ci-cd-strategy.md).
- **Calidad**: estrategia de pruebas por nivel (test-strategy.md) y verificación de documentación en cada PR.
- **Convenciones**: Conventional Commits y kebab-case en documentación.

---

## 7. Plan de trabajo y cronograma

El plan se organiza en fases alineadas a la planificación del proyecto (cronograma):

| Fase | Descripción | Entregables principales |
|---|---|---|
| 1. Habilitación técnica | Puesta en marcha del repo, CI/CD, estructura de paquetes, entorno dev | Repositorio, pipeline base, esqueleto del backend |
| 2. Módulo Security | Cuentas, roles, permisos, login JWT, auditoría | API de autenticación y administración |
| 3. Módulo Parameterization | Catálogos configurables | CRUDs de catálogos y administración web |

| Fase | Descripción | Entregables principales |
|---|---|---|
| 4. Módulo Device Management | Alta, asignación y configuración de dispositivos | API de dispositivos, gestión en web y app |
| 5. Módulo Telemetry Service | Ingesta de eventos, evidencia y alertas; sincronización | API de telemetría, agente edge, consulta de eventos |
| 6. Módulo Monitoring | Notificaciones de eventos críticos | Notificaciones push e in-app |
| 7. Módulo analítico | Línea de tiempo, métricas, resumen IA, reportes | Módulo analítico completo |
| 8. Edge (visión) | Detección de fatiga/somnolencia y alertas locales | Prototipo funcional del dispositivo |
| 9. Calidad y despliegue | Pruebas de integración, QA, despliegue qa/main | Sistema desplegado y documentación final |

> **Nota (2026-08-20):** el resumen IA se entrega al final del MVP, con la menor prioridad dentro del módulo analítico (ver scope-declaration).

Las duraciones y fechas por fase se detallan en el cronograma del proyecto y se ajustan en cada sprint.

---

# 8. Recursos requeridos

## 8.1 Recurso humano (perfiles propuestos)

| Perfil | Responsabilidades |
|---|---|
| Líder técnico / Arquitecto | Arquitectura, decisiones técnicas (ADRs), revisión de código |
| Desarrollador backend | Módulos del monolito, API REST, integraciones |
| Desarrollador frontend | Portal web (React JS) y app móvil (React Native) |
| Desarrollador edge / visión | Análisis de visión en Python, agente de sincronización |
| Tester / QA | Pruebas por nivel, criterios de aceptación |
| Documentador técnico | Documentación alineada a la gobernanza |

## 8.2 Recursos tecnológicos

| Recurso | Detalle |
|---|---|
| Infraestructura cloud | Servidores/instancias para API, base de datos y almacenamiento multimedia (proveedor a definir) |
| Dispositivos edge | Prototipos Raspberry Pi + cámara + altavoz para desarrollo y pruebas de campo |
| Herramientas | GitHub (repositorios, Actions), gestor de dependencias (Maven), herramientas de prueba y QA |
| Licencias | Software de código abierto (sin costos de licencia) salvo servicios cloud facturados por uso |

---

# 9. Estimación de costos

> Los montos son **referenciales y a validar** según el contexto del proyecto, la región y el proveedor de nube. No constituyen una cotización cerrada.

| Concepto | Estimación referencial | Supuestos |
|---|---|---|
| Infraestructura cloud (mensual) | Baja (plan inicial): USD 20–60 / mes | Instancias pequeñas, PostgreSQL gestionado, almacenamiento multimedia inicial |
| Dispositivos edge (prototipo por unidad) | USD 100–180 / unidad | Raspberry Pi + cámara + altavoz + accesorios |
| Herramientas de desarrollo | USD 0 | Stack 100 % open source; GitHub gratuito/estudiante |
| Licencias de software | USD 0 | Sin licencias propietarias en el stack propuesto |
| Capacitación y soporte | Incluido en el desarrollo | Transferencia de conocimiento en cada fase |

Los costos de recurso humano dependen de la modalidad de vinculación del equipo y se cuantifican en el plan de negocio del proyecto.

# 10. Riesgos y plan de mitigación

| Riesgo | Impacto | Mitigación |
|---|---|---|
| Conectividad intermitente en el vehículo | Pérdida de eventos | Cola local, sincronización por lotes e idempotencia |
| Precisión insuficiente de la detección de somnolencia | Falsas alertas o no detección | Calibración por `device_config`, umbrales configurables y validación con datos reales |
| Retención y volumen de evidencia multimedia | Crecimiento del almacenamiento | Política de retención configurable y almacenamiento por capas |
| Complejidad del edge (hardware/visión) | Retrasos en iteraciones | Prototipos tempranos, pruebas de campo, alcance acotado |
| Cambios en requisitos | Desvío del alcance | Backlog priorizado, actas y control de cambios documentado |
| Dependencia del proveedor de nube | Migración compleja | Stack portable (contenedores/estándares) y datos en PostgreSQL |

# 11. Entregables

| Entregable | Descripción |
|---|---|
| Código fuente | Backend (Java/Spring Boot), portal web (React JS), app móvil (React Native) y edge (Python) |
| Base de datos | Esquema versionado con Liquibase y datos de catálogos |
| Documentación técnica | Arquitectura, diseño, modelo de datos, API (OpenAPI), manuales |
| Documentación de proceso | Actas, cronograma, plan de pruebas y reportes |
| Despliegues | Entornos dev/qa/main configurados con CI/CD |
| Prototipo edge | Dispositivo funcional con detección y alertas locales |

## 12. Capacitación, soporte y mantenimiento

- **Capacitación**: sesión de transferencia por fase y guía de adopción (ver 14-training-and-adoption).
- **Soporte durante el desarrollo**: seguimiento en actas y foro del equipo.
- **Mantenimiento posterior**: evolución bajo el mismo flujo de ramas y gobernanza; los cambios de diseño se registran como ADRs.

## 13. Condiciones y supuestos

1. El proyecto utiliza exclusivamente software de código abierto en el stack propuesto.
2. Los prototipos de hardware (Raspberry Pi, cámara) son provistos por el proyecto para desarrollo y pruebas.
3. El servicio cloud se dimensiona para la carga inicial; su escalado se revisa al cierre de cada fase.
4. Los cambios de alcance se gestionan mediante actas y control de cambios.
5. La precisión de la detección de somnolencia se valida con datos de campo antes de declarar el módulo edge como estable.

*Propuesta técnica alineada con la propuesta inicial, la arquitectura vigente (ADR-001/002/003) y el modelo de datos (20 entidades).*